

캡스톤디자인 15주 강의계획서

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당한다. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 된다. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB·Simulink 가 처음이라면 6주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행한다. Onramp 는 무료이며 각각 몇 시간이면 끝난다.

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

VRX 기반 자율운항보트(USV) 제어 시스템 설계 및 자율임무 구현

- WSL2 · ROS 2 Humble · Gazebo Garden/VRX · MATLAB Simulink · AI 코딩 에이전트

항목	내용
과목명	캡스톤디자인 (Capstone Design)
학기	2026학년도 2학기
대상	학부 4학년 (자율운항시스템공학과)
선수 지식	자동제어 이수 권장. ROS 2 · Linux · Gazebo 무경험자 기준으로 설계
운영	15주 × 주 3시간 연강 (이론 + 실습 통합형)
담당	윤원근 교수 · 인공지능 필드 로보틱스 연구실 (AI Field Robotics Lab)
평가	중간고사 없음 — 8주차 중간 발표 + 15주차 기말 Term Project

1. 과목 개요 및 설계 방침

1.1 이 과목이 다루는 것

- 전반부 — Gazebo의 실제 운동모델 위에서 **Simulink 제어기로 배를 움직임**
- 후반부 — VSCode의 **AI 코딩 에이전트**로 LiDAR 충돌회피 · 자율 도킹 · 영상처리 노드를 직접 구현
- 기말 — 둘을 하나의 자율임무로 통합
- 시뮬레이션은 목적이 아니라 수단임
 - VRX는 WAM-V의 유체력 · 파랑 · 바람 · 추진기 동특성을 물리엔진 위에서 계산
 - 여기서 검증된 제어기와 인지 알고리즘은 **실선으로 이식 가능한 형태로** 남음

- 학기 말 산출물은 보고서가 아니라 **동작하는 자율운항 소프트웨어 스택**
- 예제마다 **오프라인 운동모델로 먼저**, VRX(Gazebo)는 그다음
 - 3주차 1-8 절의 WAM-V 3자유도 운동모델(**SimpleHydrodynamics** 계수 그대로)을 3~10주차 오프라인 모델이 함께 씀
 - 오프라인에서 예측하고 계인을 맞춘 뒤, 같은 제어기를 VRX 에 물려 확인하는 순서
- 기호는 전 과목 공통 — 북·동 위치 x, y (코드 x_n, y_n), N 은 요 모멘트 전용

1.2 설계 원칙 세 가지

① Motivation-first — 1주차부터 배를 띄운다

- ROS 2 무경험자에게 3주를 순수 튜토리얼로 쓰면 흥미가 급락함
- 환경구축 단계에서도 **매 시간 눈에 보이는 결과**를 만듦
 - 터미널 분할 → 시뮬레이터 실행 → 배의 움직임
- 필요한 개념은 그때그때 주입

② 정량화가 곧 평가 기준이다

- "돌아갔다"는 결과물로 인정하지 않음
- 모든 실습 과제는 **지표를 정의하고 측정한 값**을 제출
- **실패한 실험도 원인 분석이 있으면 만점**
- 성공했으나 측정하지 않은 결과는 감점
- 목적: 동작하는 물건이 아니라 **설계 근거를 설명할 수 있는 엔지니어**

③ AI 에이전트는 도구로 쓰되 책임은 학생이 진다

- 후반부 인지 알고리즘 구현에 VSCode + Claude 에이전트를 적극 활용
- 평가 대상: "AI를 썼는가"가 아니라 **"AI 출력을 검증했는가"**
- 상세는 6장

1.3 세 개의 층위

Phase	주차	내용
I. 기반	1~4주	WSL2 · ROS 2 · Gazebo/VRX 개발환경 구축, 좌표계·TF, WAM-V 모델 구조, WAM-V 3자유도 운동모델 · 센서 모델
II. 제어	5~10주	AI 에이전트 환경, Simulink-ROS 2 연동, USV 운동모델, heading·속도 제어, LOS 유도, 로이터링, Stateflow 미션, 추력 배분·동적위치유지(DP)
III. 인지	11~13주	LiDAR 포인트클라우드 처리, 충돌회피, 영상처리, LiDAR-카메라 융합
IV. 통합	14~15주	도킹 제어, 미션 매니저, 반복시험과 성능 정량화, 최종 발표

1.4 매주 3시간 반복 구조

구간	시간	내용
이론	50분	그 주 임무를 푸는 데 필요한 제어·신호처리 이론. 칠판 수식 유도 포함
실습	110분	VRX / Simulink / ROS 2 직접 구현. 팀 단위, 조교 1명 배치
팀 스كر럼	20분	결과 비교, 실패 원인 공유, 다음 주 과제 및 Term Project 진척 점검

2. 기술 스택

항목	채택	근거 및 주의
OS	WSL2 + Ubuntu 22.04	학생 개인 노트북(Windows) 기준. WSLg로 GUI 동작, 별도 X 서버 불필요
ROS 2	Humble Hawksbill	LTS(2027.5까지). 자료·패키지 최다. MATLAB ROS Toolbox 지원 안정적
Gazebo	Garden (gz-garden)	VRX humble 브랜치 요구 버전. Garden은 비-LTS(2024.11 EOL)이나 기존 검증 자료를 그대로 살리기 위해 유지
시뮬레이터	VRX 2.4.1 (humble 브랜치)	clone 후 <code>git checkout humble</code> 필수
제어기 설계	MATLAB R2024b + Simulink + ROS Toolbox	제어기 설계·튜닝·자동 코드생성의 주력 도구
인지·미션	Python 3 (rclpy)	포인트클라우드·영상처리·미션 매니저는 ROS 2 네이티브로
개발환경	VSCode + WSL Remote + Claude Code	후반부 인지 알고리즘 구현
시각화	RViz2 + Mapviz (+ mapproxy Docker)	Mapviz는 위경도 항적 확인용
형상관리	GitHub Organization	팀별 레포 + 공용 패키지

버전 정책: Humble + Garden 조합은 "지금 가장 자료가 많지만 수명이 짧은" 선택이다. 학기 중에는 속도를 우선해 이 조합으로 진행하고, 최신 조합(Jazzy + Harmonic) 이관은 방학 과제로 남긴다.

2.1 하드웨어 요구사항

항목	최소	권장
OS	Windows 10 2004 이상	Windows 11
CPU	4코어 x86-64	i5 / Ryzen 5 이상
RAM	8 GB	16 GB 이상 (VRX + Gazebo + MATLAB 동시 구동)
저장공간	40 GB 여유 (학기 전체. 2·3주차 설치 시점에는 20 GB 이상)	SSD
GPU	내장 그래픽	외장 GPU (Gazebo 렌더링)

사양 미달 학생은 1주차 과제 제출 시 신고하면 연구실 워크스테이션 원격 접속(SSH / NoMachine)을 배정한다.

3. 재사용 자산

- 이 과목은 백지에서 시작하지 않는다. 연구실이 축적한 아래 자산을 주차별로 투입한다.

3.1 Simulink 모델

파일	내용	추진기	투입
X_pose_config_test_2022a.slx	제어기 없는 순수 ROS 2 인터페이스 / 프레임 변환 테스트	무관	참고 (수업 미사용)
GPS_INS_integration.slx	GPS + IMU 융합 → /gnss_ins_odom 발행	무관	참고 (수업 미사용)
VRX_tilt4_controller_unberthing.slx	이안 + 위치/헤딩 제어	4개	7~8주 참고
VRX_tilt4_controller_full.slx	Stateflow 미션 FSM + 추력배분 + LOS + 상태추정	4개	9-13주 참고
VRX_manual_control.slx	조이스틱 수동조종	3개	참고 (수업 미사용)
VRX_control_final_docking_2022.slx	내부에 WAM-V 동역학 모델 포함 → Gazebo 없이 오프라인 실행 가능	3개	예비

✕ 기존 Simulink 자산은 4추진기 · 3추진기용이다

본 과목은 2추진기를 쓰므로 그대로 돌아가지 않는다. 다만 버릴 필요는 없다.

부분	재사용 여부
상태추정 (fuseIMUGPS , LLA→NED, ENU→NED)	그대로 재사용
LOS 유도 (waypoint_cmd)	그대로 재사용
헤딩 · 속도 제어기	그대로 재사용
Stateflow 미션 FSM	그대로 재사용
추력배분 (rpi_otter)	교체 — 3×8 의사역행렬 → 2×2 차등 역변환

즉 **배분 블록 하나만 바꾸면** 나머지 구조는 그대로 쓸 수 있다.
이것이 제어기를 모듈로 나눠 설계하는 이유이기도 하다 (7주차에 다름).

★ 이번 학기 tilt4 자산은 읽기 자료로만 쓴다

수업에서 실행하는 모델은 전부 2추진기용으로 새로 만든 것이다.

tilt4 는 "추진기가 더 많으면 어떻게 되나"를 보여 주는 **코드 리딩 자료**이고,
VRX_tilt4_controller_full.slx 의 **Stateflow 미션 FSM** 구조는 14주차에서 참고한다.

이 학기에 새로 만든 배포 모델

파일	내용	추진기	투입
<code>W02_simulink/W02_1~3*.slx</code> (3개)	turtlesim 자세 수신 → 오프라인 목표 자세 제어 → 실제 turtlesim 구동	무관	2주
<code>W03_simulink/W03_0~4*.slx</code> (5개)	좌표 변환 검산 → 운동모델 버튼 조종 (VRX 불필요) → 센서 NED 변환 → 추력 부호 확인 → 버튼 조종	2개	3주
<code>W04_simulink/W04_0~2*.slx</code> (3개)	운동모델 + GPS/IMU 센서 모델 (VRX 불필요) → 센서 수신 주기 · QoS 불일치를 Simulink 에서 재측정	2개 (<code>W04_0</code>) · 무관	4주
<code>W06_0_simulink/SB1~SB13_{todo,done}.slx</code> (26개)	Simulink 속성 입문 빈칸본·완성본 13쌍	무관	6주 보충
<code>W06_simulink/W06_P1~P3*.slx</code> (3개)	PID 입문 — 전달함수 스텝응답, PID 직접 조립, 오프라인 배 속도 제어	무관	6주
<code>W06_simulink/W06_1~5*.slx</code> (7개)	직진 → 선회 → 헤딩 제어 → 속도+헤딩 내부루프 (VRX 4개) + 오프라인 3개 (헤딩 · 내부루프 쌍둥이, 개루프 WAM-V)	2개	6주
<code>W07_simulink/W07_0_offline.slx</code>	Gazebo 없이 도는 3자유도 WAM-V . 조류 실험 가능	2개	7주
<code>W07_simulink/W07_1_vrx.slx</code>	같은 유도-제어기, 운동모델만 Gazebo	2개	7주
<code>W08_simulink/W08_0_offline.slx</code>	벡터필드 로이더링. 방향·속도·반경 변경	2개	8주
<code>W08_simulink/W08_1_vrx.slx</code>	같은 유도, 운동모델만 Gazebo	2개	8주
<code>W09_simulink/W09_0_offline.slx</code>	Stateflow 미션 — WP → 로이더 → WP → 종료	2개	9주
<code>W09_simulink/W09_1_vrx.slx</code>	같은 미션, 운동모델만 Gazebo	2개	9·13 주
<code>W10_simulink/W10_0_offline.slx</code>	방위추진기 2기 DP — 추력 배분 · 기준모델 · 바람·파랑 · 파랑 필터	2개 (방위)	10주
<code>W10_simulink/W10_1_vrx.slx</code>	같은 DP, 운동모델만 Gazebo. <code>thrusters/*pos</code> 로 방위각 발행	2개 (방위)	10주

- 5주차는 Simulink 모델 대신 `W05_python/wamv_sim.py` — VRX 와 같은 토픽을 내는 파이썬 운동모델. 학생 노드를 VRX 전에 먼저 물림
- 7~9주차 모델은 **내부루프·추진기·운동모델·게인**을 공유한다. 주차마다 유도단만 바뀐다
 - 내부루프 구조는 6주차 4단계 그대로, 게인만 7주차 값(헤딩 P-D 400/200, 속도 PI 300/100)
- 두 W07 모델의 운동모델 계수는 Gazebo `SimpleHydrodynamics` 에서 그대로 가져옴
- 검증 (2026-09-19): 230 s · 312 m 주행 후 정착 후 평균 $|y_e|$ 0.702 vs 0.784 m, 정상상태 속도 1.500 vs 1.501 m/s. 궤적 평균 이격은 0.95 m 이나 같은 설정 세 번에 0.28 · 2.59 · 0.95 m 로 흔들림 (위상 오차, §9 참조)

3.2 MATLAB 스크립트

- `VRX_SHIFT_MINI_Full.m` — 마스터 실행 스크립트. 위성지도에서 `ginput` 으로 7점 미션(이안 → WP×3 → DP → 접근 → 도강)을 클릭 정의하고, 추진기 기하와 전체 제어 게인을 설정한 뒤 모델을 실행한다.
- `WAMV_USV_Control_Allocation.m` — 추력배분 교재. **2 방위추진기** 기준($\pm 45^\circ$ 사용가능 콘, 데드존, 역추력 매핑, Monte-Carlo ACS 그림). 본 과목 구성(고정 2추진기)과는 다르므로 **비교·대조 자료**로 쓴다.
- `USV_display.m` / `USV_draw.m` / `circle_draw.m` — 위성지도 위 실시간 항적 애니메이션.

3.3 ROS 2 워크스페이스 (`src/`)

커스텀 월드 — Term Project의 무대

월드	출처	구성
<code>sydney_regatta.sdf</code>	upstream	기본 해역
<code>sydney_regatta_ca.sdf</code>	연구실 추가	적색 마커부표 36개 장애물 필드
<code>sydney_regatta_ca_v2.sdf</code>	연구실 추가	색상 혼합 소규모 필드
<code>scan_dock_deliver_CA.sdf</code>	연구실 추가	부표 80개 + 도킹 플랫폼
<code>scan_dock_deliver_full.sdf</code>	연구실 추가	<code>custom_docking_station</code> (bay1/bay2/bay3)

✕ 연구실 추가 파일은 `git clone` 으로 받아지지 않음

2026-09-03 에 osrf/vrx humble 브랜치(커밋 `dc30ed8d`)와 직접 대조해 확인함.

월드 4개 + `competition4docking.launch.py` + `custom_docking_station` 모델을

별도 배포 패키지로 학생에게 전달해야 함 → [WSL-VRX-환경구축](#) §5.5

Python 노트

파일	내용
<code>vrx_control/collision_avoidance_v2.py</code>	±30° 단순 회피 — 나쁜 baseline 교보재
<code>vrx_control/collision_avoidance_kaboat.py</code>	LaserScan → 슬라이딩윈도우 장애물 밀도 → 비용 기반 안전방위 선정
<code>vrx_control/wamv_pid_control_v2.py</code>	UTM 좌표 웨이포인트 PID
<code>ros2_simulink/auto_berthing_seek.py</code>	복셀 다운샘플 → RANSAC 평면분할 로 도크 벽면 법선 추정 → <code>/docking_info</code>
<code>pointcloud_to_laserscan/</code>	PointCloud2 → LaserScan 변환

3.4 WAM-V 추진기 구성 — 기본 후방 2추진기

★ 이 수업은 VRX 기본 WAM-V(후방 2추진기) 를 쓴다

`competition.launch.py` 가 스폰하는 기본 구성이 그대로 이것이다. 별도 설정이 필요 없다.

```
left  (-2.373776,  1.027135, 0.318237)  장착각 0°
right (-2.373776, -1.027135, 0.318237)  장착각 0°
```

- 정의 파일: `vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_aft_thrusters.xacro`
- 토픽: `/wamv/thrusters/{left,right}/thrust` (`std_msgs/Float64`)
- 좌우 반폭 **1.027 m** → 두 추진기 간격 2.054 m
- 조인트는 회전 가능(`revolute` , ±180°)하고 `.../pos` 토픽도 있으나, 각도는 0으로 고정하고 추력만 쓴다

차동 추력배분 (2×2)

```
X = F_L + F_R          전진력
N = (F_L - F_R) × 1.027  요모멘트 (NED, 양수면 우선회)

역변환
F_L = X/2 + N / (2 × 1.027)
F_R = X/2 - N / (2 × 1.027)
```

- 의사역행렬이 필요 없는 **정칙 2×2 역변환**
- 과소구동** — 3자유도를 원하지만 독립 입력은 2개. **sway** 를 직접 제어할 수 없다
- 옆으로 가려면 선수를 돌려야 함 → 12주차 충돌회피 설계의 핵심 제약

선택 이유

항목	4방위추진기(tilt4)	기본 2추진기
배분	3×8 가중 의사역행렬 + 각도포화 반복	2×2 역변환
학습 부담	높음	낮음 — 제어 논리에 집중 가능
VRX 기본값	별도 설정 필요	그대로 동작
KABOAT 실물	일부만 해당	대부분이 차동 추진

3.5 참조 문헌

주교재 (7~9주) — Jin, Li, Yang, Shan, Zhang (2025), *Construction of Simulation System for USV Motion Control and Design of Multi-Mode Controllers Based on VRX and Simulink*, **Applied Sciences** **15(8):4213**. (오픈액세스, 강의자료 폴더 보유)

- 이 논문은 Ubuntu 22.04 + ROS 2 Humble + MATLAB 환경에서 WAM-V 3자유도 Fossen 모델, PI 속도제어 + PID heading제어 (극배치) + 적분 LOS 유도, `robot_localization` EKF를 구성한 사례로, 본 과목 전반부와 구조가 거의 일치한다.
- Figure 3 (p.7) — 전체 운동제어 블록선도. **본 과목의 마스터 아키텍처 그림으로 사용**
- Table 1 (p.12) — WAM-V 파라미터 19종. 7주차에 배포
- Table 3 (p.14) — 제어기·기준모델 게인. 8주차 튜닝 출발점
- Table 4 / Figure 14 (pp.16-17) — 4가지 외란 시나리오 시험 매트릭스. 14주차 반복시험 설계의 본보기

한계: 이 논문에는 LiDAR·카메라·충돌회피·도킹이 없다. 11주차 이후는 별도 자료로 다룬다.

50-원 본 자료 / 에 있는 문헌

파일	투입
Fossen 2024 Lecture on 2D and 3D path-following control.pdf	7주차 주교재. LOS·ILOS 유도
Handbook Of Marine Craft Hydrodynamics And Motion Control(2nd) @.pdf	참고. 운동방정식·유도·제어 전반
1. 동역학 모델.pdf / .pptx	참고. 운동방정식 유도
2. USV 제어기 설계.pdf / .pptx	7주차 37~58쪽 두 오차 · LOS · lookahead (59~71쪽 ILOS 는 범위 밖)
3. ODE and Dynamic System Simulation.pdf	참고. Simulink 수치적분
4. USV Practical 제어기 설계.pdf / .pptx	9주차 주교재. 74~98쪽 이산 PID·안티와인드업
0. 선형제어기 기본 설계.pdf	참고. 8주차 게인 조율의 배경
5. System Identification.pptx	참고. Nomoto 식별
Ch2.pdf , Ch3.pdf	Fossen 교과서 발췌
Term Project_r1.pdf , 2025 KABOAT 임무설명_r1.docx	Term Project 참고

볼트 밖 코드 자산

경로	내용	투입
선형 제어 시스템 /MATLAB_source/Examples/MSS/	Fossen MSS 툴박스 전체. SIMULINK/mssSimulinkDemos/demoOtterUSVHeadingControl_P_D_Waypoint.slx 가 LOS/atan2/LOS 를 모드로 제공. 7주차 본문이 인용하는 것은 MSS GitHub 판의 demoOtterUSVPathFollowingHeadingControl.slx · demoOtterUSVPathFollowingCourseControl.slx	7주차 원본
선형 제어 시스템 /.../PID_Waypoint_Tracking_WAMV/	LOSchi.m , Simple_guidance.m , ssa.m	7주차 배포본으로 복사

보조 자료: VRX Wiki 튜토리얼

4. 주차별 강의 계획

Phase I — 개발환경 구축과 ROS 2 / Gazebo 기초

1주차 · 과목 개요, USV 자율운항 개관, WSL2 구축

이론 (50분)

- 과목 목표와 Term Project 정의 — 학기 말에 무엇을 만들 것인가
- 자율운항 USV 개관: 인지 → 판단 → 제어 파이프라인
- KABOAT 5대 임무 / VRX 8개 Task 소개
- 시뮬레이션 기반 개발의 의미: MILS → HILS → 실선 시험
- 팀 편성 (4~5명 × 3~4팀), 역할 분담, 협업 도구 안내

실습 (110분)

- 관리자 PowerShell에서 `wsl --install -d Ubuntu-22.04`
- 리눅스 필수 명령어: `cd` `pwd` `ls` `apt` `source` `chmod` `mkdir` `rm`
- `terminator` 설치 및 분할 단축키 (`Ctrl+Shift+E` 좌우 / `Ctrl+Shift+O` 위아래) — 2026-09-03 설치본에서 확인
- 노트북 하드웨어 사양 점검

과제 — 설치 완료 스크린샷 + 노트북 사양표 제출. 사양 미달자는 이때 신고

자료 — [WSL-VRX-환경구축](#) §1~2

2주차 · ROS 2 Humble 설치, 노드 / 토픽 / DDS

이론 (50분)

- ROS 2 구조: 노드, 토픽, 서비스, 액션, 파라미터
- DDS와 `ROS_DOMAIN_ID` — 같은 네트워크의 다른 학생과 토픽이 섞이는 이유
- QoS 프로파일 (Best Effort vs Reliable, Depth)이 센서 데이터에 미치는 영향
- 메시지 타임스탬프와 지연(latency), 표본화 주기 불일치

실습 (110분)

- ROS 2 Humble 설치 (locale → 저장소 등록 → `ros-humble-desktop`)
- VS Code 설치와 WSL 연결

- turtlesim 으로 토픽·서비스·액션·파라미터 네 가지 통신 조작
- `ros2 run` / `ros2 topic` / `ros2 node` / `ros2 interface` 조사 명령어
- `rqt_graph` · Topic Monitor · `rqt_console` · RViz2 로 상태 확인
- colcon 워크스페이스·패키지 생성, rclpy 퍼블리셔·서브스크라이버 작성
- QoS 불일치 재현 실험 (BEST_EFFORT 발행 ↔ RELIABLE 구독)
- 중계 노드 `turtle_pose_relay` — MATLAB 이 모르는 메시지를 표준 형식으로 옮김
- Simulink 로 turtlesim 목표 자세 제어 (`W02_1~3` , 모델 3개. MATLAB 없으면 6주차 뒤로)

과제 — 지정된 잡음 특성(바이어스 + 가우시안 잡음)을 갖는 가상 IMU를 **50 Hz로 발행하는 노드**와, 이를 **10 Hz로 구독·재표본화하는 노드**를 작성하고 두 신호를 겹쳐 그린 그래프 제출

자료 — [WSL-VRX-환경구축](#) §3

3주차 · Gazebo Garden + VRX 설치·실행, 좌표계와 TF2

이론 (50분)

- 시뮬레이터 아키텍처, 물리엔진, sim time
- SDF · URDF · Xacro의 차이와 역할
- VRX의 파랑·바람·유체력 플러그인
- 선박 6자유도 운동 (surge, sway, heave, roll, pitch, yaw)
- **ENU vs NED 좌표계** — ROS는 ENU, 선박공학은 NED
- 회전 표현: 오일러각, 쿼터니언, DCM과 짐벌락
- **WAM-V 3자유도 운동모델** — 선체 축 · 추진기 위치 · 질량/요 관성 · 항력 계수 (`SimpleHydrodynamics`) · Fossen 형식 운동방정식 · `EOM` 코드

실습 (110분)

- Gazebo Garden 설치 (`gz-garden` , `ros-humble-ros-gz-garden`)
- VRX 설치 — `git clone` 후 `git checkout humble` 필수, `colcon build --merge-install`
- `ros2 launch vrx_gz competition.launch.py world:=sydney_regatta` 실행
- 명령어(`ros2 topic pub`)와 키보드(`wamv_teleop_key`)로 WAM-V 직진·선회 (차동 추진)
- TF 트리 확인 (`view_frames` , `tf2_echo` , RViz2)
- Simulink 로 좌표 변환 확인 (모델 5개)
 - 3-1 `W03_1_frame_check` — 변환식 검산 (VRX 불필요)
 - 3-2 `W03_0_offline` — 1-8 절 운동모델을 버튼으로 먼저 조종 (VRX 불필요). `W03_offline_run.m` 으로 손계산과 대조
 - 3-3 `W03_2_vrx_nav` → 3-4 `W03_3_vrx_drive` (3-2 와 같은 추력으로 오프라인 대조표) → 3-5 `W03_4_teleop` 같은 버튼으로 VRX 조종
 - 버튼 조종 창(`W03_teleop_plot`)에 좌 · 우 추력 F_L , F_R 그래프

과제 — `frame_converter` 노드 (GPS LLA → NED, IMU 쿼터니언 → NED 오일러각) 작성 + 검증 3종 (왕복 변환 오차, 방향 확인, 선수각 $\pm 180^\circ$ 정리)

자료 — [WSL-VRX-환경구축](#) §4~5

자산 — `W03_simulink/` (모델 5개 + `W03_offline_run.m` · `W03_vrx_run.m`)

4주차 · VRX 심화: 토픽 전수조사, WAM-V 모델 구조, Mapviz

이론 (50분)

- WAM-V 형상과 센서 구성: GNSS, IMU, 3D LiDAR(16빔), 카메라 3대
- **센서 배치가 인지 성능에 미치는 영향** — FOV, 폐색(occlusion), 근거리 사각지대
- **후방 2추진기 배치**와 차동 추진으로 방향을 만드는 원리
- `wamv_aft_thrusters.xacro` 구조 읽기
- **센서 모델** — 운동모델의 참값 위에 GPS(20 Hz, 안테나 장착 위치) · IMU(100 Hz, 자이로 잡음 · 바이어스)를 엮음

실습 (110분)

- 토픽 전수조사 (`ros2 topic list` / `hz` / `info --verbose`), 센서별 정상 출력 확인
- `wamv_gazebo.urdf.xacro` 복사본 수정 + `urdf:=` 로 LiDAR 높이 변경, RViz2 점군 비교
- **Mapviz + mapproxy Docker** 설정 → 위성지도 위 위경도 항적 표시

- VRX 과제 월드 실행과 `/vrx/task/info` 채점 토픽, `ros2 bag` 기록·재생
- Simulink 로 재측정 (모델 3개)
 - 2-8-0 `W04_0_offline` — 운동모델 + GPS/IMU 센서 모델로 먼저 (VRX 불필요). `W04_offline_run.m` 으로 주기 · 자이로 잡음 · 안테나 거리 확인
 - 2-8-1 `W04_1_sensor_rates` (수신 주기, VRX) · 2-8-2 `W04_2_qos_test` (QoS 불일치)

과제 (중요) — VRX 토픽 전수조사표 작성 (토픽명, 메시지 타입, 방향, 발행 주기, `frame_id`, QoS) + 센서 배치 변경 실험 전후 비교. 이 표는 이후 전 주차에서 참조하는 팀 공용 문서가 된다.

자료 — [WSL-VRX-환경구축](#) §6~7

자산 — `W04_simulink/` (모델 3개 + `W04_offline_run.m` · `W04_rates_run.m`)

Phase II — Simulink 제어기 트랙

5주차 · VSCode + Claude AI 에이전트 개발환경, 첫 ROS 2 제어 노드

이론 (50분)

- AI 코딩 에이전트의 동작 원리와 한계 — 무엇을 잘하고 무엇을 틀리는가
- 에이전트가 만든 코드를 검증하는 법: 단위 테스트, rosbag 재생 검증, 경계조건 확인, 좌표계 부호 검증
- `CLAUDE.md` 로 프로젝트 컨텍스트를 주는 법
- 학술적 정직성 정책 (→ 6장) 안내

실습 (110분)

- VSCode + WSL Remote 연결, ROS 2 확장 설치
- Claude Code 설치 및 인증
- 팀 레포에 `CLAUDE.md` 작성 — ROS 2 버전, 토픽 규약, 좌표계 규약, 코딩 규칙
- 에이전트로 웨이포인트 PID 노드 생성 → 직접 리뷰·수정 → 세 겹 검증(단위 테스트 · rosbag 재생 · 경계조건)
- E-4 — 노드를 `W05_python/wamv_sim.py` (VRX 와 같은 토픽을 내는 파이썬 운동모델)에 먼저 물리고, 그다음 VRX
- MATLAB Agentic Toolkit(MCP)으로 Claude 에서 MATLAB-Simulink 조작

과제 — 에이전트 생성 초안과 학생 수정 최종본의 **diff + 수정 사유 3가지 이상** 제출

자산 — `vrx_control/wamv_pid_control_v2.py` 를 비교용 정답지로 배포 · `W05_python/wamv_sim.py` (오프라인 시뮬레이터)

6주차 보충 · Simulink 속성 입문 (3시간 × 2회)

6주차 본편 전에 두 번에 나눠 진행한다. ROS-Gazebo-VRX 없이 MATLAB 하나로 한다.

구성

- 1일차 A~F (문법) — 첫 모델과 솔버, MATLAB Function, Subsystem-Goto/From, 버스, PID-포화-안티와인드업, 파라미터·로깅·Data Inspector
- 2일차 G~M (7~9주차 모델을 읽기 위한 블록) — Mux·Demux·Selector, Unit Delay·대수 루프, Integrator·Transfer Fcn·솔버, Switch·Multiport Switch·Stop, Enabled-Triggered Subsystem, Mask·라이브러리·코드로 모델 생성, Stateflow 첫걸음

실습 — `W06_0_simulink/` 의 빈칸본 `SB1` ~ `SB13` `_todo` 를 채우고 `_done` 과 대조 (모델 26개 = 13쌍)

과제 — W06-0A (1일차 여섯 모델과 측정, 6주차 수업 전), W06-0B (2일차 일곱 모델과 측정, 7주차 수업 전). **(무배점 연습)** — 5장의 "주차별 실습 과제 (12회)" 에 넣지 않음

자료 — [W06_0 Simulink 기초](#)

6주차 · Simulink ROS 2 연동과 첫 제어기

이론 (50분)

- Simulink 와 ROS 2 의 연결: Subscribe / Blank Message + Bus Assignment / Publish, Domain ID 정합
- 속도 `u` 를 얻는 법 — ground truth odometry
- 제어 구조: 헤딩 루프 + 속도 루프 → 차동 추력배분
- 실시간 페이싱 (Simulation Pacing) — 켜지 않으면 배가 움직이지 않는 이유
- 개루프와 피드백, PID 의 세 갈래 ($P \cdot I \cdot D$)

- heading 게인의 출처 — Nomoto 1차 모델과 극배치
- heading 제어는 **P-D 요각속도 되먹임** — 오차를 미분하지 않는 이유 (H절)

실습 (110분)

- A · B — ground truth odometry 커기 (urdf 복사본 + `urdf:=`), MATLAB ↔ WSL 토픽 확인
- C · D — `W06_1_straight` 직진, `W06_2_turn` 직진·우선회·좌선회
- E~G — **PID 입문** (Gazebo 불필요): `W06_P1_pid_step` 게인 역할, `W06_P2_pid_byhand` 직접 조립·미분 필터·안티와인드업, `W06_P3_boat_speed` 오프라인 배 속도 제어
- H — heading 제어. 오프라인 쌍둥이 `W06_3_heading_offline` 먼저 → `W06_3_heading` VRX (`W06_step_compare(3)` 이 둘을 나란히)
- I — 속도 + heading (inner loop 완성). 오프라인 쌍둥이 `W06_4_inner_loop_offline` 먼저 → `W06_4_inner_loop` VRX (`W06_step_compare(4)`)
- J — `W06_5_offline` 오프라인 WAM-V (2단계 개루프의 쌍둥이), `W06_vrx_record.m` 로 VRX 와 대조

과제 — 과제 6 첫 제어기 응답 측정: ① PID 게인 역할 (P / PD / PID, 미분 필터 `Nf2`, 안티와인드업 `Kb`) ② heading 60° 스텝응답 3 조건 ③ 속도 스텝응답 PI vs P ④ 선회 중 속도 저하 (커플링) ⑤ 분석

자산 — `W06_simulink/` (모델 10개: VRX 연동 4 + 오프라인 3 + PID 입문 3, 비교 스크립트 `W06_step_compare.m`)

7주차 · 웨이포인트 유도 — atan2 와 LOS

이론 (50분)

- 유도 · 항법 · 제어(GNC)의 분리 — 6주차는 제어만 했다
- 선수각 ψ / 침로각 χ / 크랩각 β — $\chi = \psi + \beta$, $\beta = \text{atan2}(v, u)$
- 가장 단순한 유도: $\psi_{\text{ref}} = \text{atan2}(\Delta y, \Delta x)$ 과 그 한계 (x 북, y 동)
- **cross-track error** $y_e = -(x - x_k) \sin \pi_p + (y - y_k) \cos \pi_p$
- **LOS 유도법칙** $\psi_{\text{ref}} = \pi_p - \text{atan}(y_e / \Delta)$
- lookahead distance Δ 와 조류 속 정상 이탈 $y_e = \Delta \tan \beta$
- 웨이포인트 전환 판정 — 수락 원 vs 경로 방향 ($d - x_e < R$), 수락반경 R
- 미션 종료 — 마지막 웨이포인트 도달 시 `gate = 0` 으로 속도 지령 0, 한 번 끝나면 계속 정지 (래치)
- 내부루프는 6주차 그대로 — heading P-D(요각속도 되먹임), 속도 PI(`u` 되먹임)

실습 (110분)

- A — `W07_setup.m` 하나로 웨이포인트·게인·조류를 정의. 실행하면 오프라인 모델이 바로 돌
- B — `W07_0_offline.slx` 구조 읽기 (Guidance → InnerLoop → Thrusters → MotionModel)
- C — atan2 ↔ LOS 전환, **조류 실험**. `W07_compare.m` 로 4조건 자동 비교
- D — $\Delta \cdot R$ · 전환 판정 바꿔 보기 (`W07_lookahead_demo.m`, `W07_crab_demo.m`). 1-7 · 1-8 절 표는 `W07_sweep.m` 로 재현
- E — `W07_1_vrx.slx` VRX 연동. `W07_vrx_run.m` 이 토픽 확인 · RTF 측정 · 페이싱 · 스폰 원점 실측 · 오프라인 대조

★ 두 모델은 운동모델만 다르다

오프라인 운동모델이 Gazebo 의 `SimpleHydrodynamics` 계수를 그대로 쓴다.

2026-09-19 재측정: 230 s · 312 m 주행 후 정착 후 평균 $|y_e|$ **0.702 vs 0.784 m**, 정상상태 속도 **1.500 vs 1.501 m/s**, 게인 변경 없음. `W07_vrx_run.m` 한 줄로 재현됨

궤적 평균 이격은 같은 설정 세 번에 0.28 · 2.59 · 0.95 m 로 실행마다 달라짐 — 페이싱이 만드는 위상 오차라 판정에 쓰지 않음

과제 — atan2 vs LOS × 조류 유·무 4조건 성능표, Δ 스위치(2/3/5/10/20/30 m × 조류 없음·0.4 m/s), 조류 방향 3종(0°/90°/180°)의 크랩각, **오프라인 ↔ VRX 대조**. 조류 세기는 전 항목 0.4 m/s

교재 — Fossen 강의노트(2024), MSS Otter 경로 추종 데모와 `GNC/LOSchi.m` · `crosstrackWpt.m`, 「2. USV 제어기 설계」 37~58 쪽

8주차 · 한 점을 중심으로 도는 로이터링

★ 8주차는 중간 발표 주간이다

평가 15% (평가와-팀운영). 수업 앞 60분에 팀별 발표(전반부 데모 + Term Project 제안서), 남은 시간에 로이터링 실습을 진행한다. 이론(50분)은 요약으로 줄이고 자료를 미리 읽어 오게 한다.

이론 (50분)

- 로이터링이 무엇이고 왜 필요한가 — 감시·대기·통신중계
- 웨이포인트로 원을 흉내내면 안 되는 이유 세 가지
- 벡터필드 — 경로를 쫓는 대신 공간에 방향을 깔아 둔다
- 논문 식 (9) 극좌표 벡터필드, 식 (13) 침로 명령
- p_c — 부호가 회전 방향, 크기가 원에 끌리는 정도
- 식 (27)·(28) 로 p_c 를 계산: $p_c = 4 \zeta_c^2 v \tau_r / r_d$
- 반경 오차가 바깥으로 남는 이유 — 지연 세 가지 (헤딩 램프 오차·모터·크랩각)
- 회전수 세기와 임무 종료

실습 (110분)

- `W08_vectorfield.m` — 네 조건의 벡터필드를 눈으로 확인
- `W08_0_offline.slx` — 오프라인 로이터링. 실행하면 2바퀴 돌고 정지
- 방향(p_c 부호)·속도·반경을 바꾸기. `use_schedule = 1` 로 도중에 바꾸기 — `W08_segments.m` 이 구간별 반경 · 속도 · 방향 표를 냄 (C-2)
- 크랩각 보상 — 반경 오차 2.14 → 1.24 m
- `W08_1_vrx.slx` — VRX 연동 (`W08_vrx_run.m`)

I 실측으로 확인한 것

- 반경 오차가 p_c 에 정확히 비례한다 (오차 ÷ p_c = 6.82 로 일정).
- 등가 지연 4.55 s = 헤딩 램프 오차 2.50 + 모터 0.30 + 크랩각 1.93 (예측 4.73).
- 2026-09-19 재측정: 정착 후 평균 반경오차 오프라인 **2.138 m** / VRX **2.265 m**, 둘 다 2.00 바퀴.
궤적 평균 이격 0.77 m (최대 1.96 m). 09-18 에는 같은 반경오차로 이격 4.96 m 였음 — 같은 원을 조금 다른 속도로 도는 위상 오차라 판정은 반경오차·회전수로 함

과제 — 두 구현(압축형 한 줄 식 ↔ 모델의 `LoiterVF`)이 같음을 유도, p_c 스윙(0.15/0.313/0.6/1.0)과 비례 관계 확인, 크랩 보상 효과의 수치적 설명, 방향·속도·반경 변경 기록, 오프라인 ↔ VRX 대조

교재 — W. Jung, S. Lim, D. Lee, H. Bang, "Unmanned Aircraft Vector Field Path Following with Arrival Angle Control" (50-원본 자료 / PDF)

자산 — `W08_matlab/` (`Guid_VF_Loiter.m` , `VF_test_code_main.m` — 연구실 원본, 읽기용)

9주차 · Stateflow 미션 — 웨이포인트와 로이터링을 잇는다

이론 (50분)

- 유한상태기계(FSM)가 왜 필요한가 — `if` 문으로 짜면 무너지는 이유
- 상태 · 전이 · 전이조건 · 진입동작
- 본 주차의 세 상태: `WpFollow` → `Loiter` → `WpFollow` → `Finish`
- 두 유도법칙을 갈아 끼우는 법. 둘 다 돌리고 출력만 고른다
- 로이터 중 웨이포인트 인덱스를 잠가야 하는 이유
- 로이터 중심을 상수로 고정해야 하는 이유
- 대수 루프 — 왜 생기고 단위지연으로 어떻게 끊는가
- 로이터를 마친 배는 경로 밖에 있다. LOS 가 다시 끌어온다

실습 (110분)

- `W09_0_offline.slx` — 웨이포인트 → WP3 에서 2바퀴 로이터 → 웨이포인트 → 종료
- Stateflow 차트 열어 상태·전이조건 읽기, 사본 `W09_my` 에 차트 직접 그리기
- `loiter_wp` , `loiter_radius` , `loiter_turns` , p_c 바꾸기
- `W09_1_vrx.slx` — VRX 연동 (`W09_vrx_run.m`)

i 실측 (2026-09-18)

- 오프라인: 로이터 진입 74.7 s → 종료 248.0 s (2.00 바퀴) → 임무 종료 314.4 s. WP 구간 평균 $|y_e|$ 2.67 m, 로이터 평균 반경오차 1.28 m
- VRX (2026-09-19, 같은 초기 선수각의 오프라인과 대조): 상태 순서·회전수 같음, 로이터 반경오차 1.213 vs 1.282 m, 임무 종료 301.1 vs 310.6 s, 궤적 평균 이격 4.78 m (최대 14.44 m) — VRX 가 9.5 s 빨리 끝난 위상 오차 (09-18 은 이격 0.83 m)

과제 — 상태 전이 기록, **인덱스 잠금**을 풀어 보고 무슨 일이 생기는지 관찰, 로이터 파라미터 스윙, **로이터를 두 번 하도록 상태기 계 직접 수정**, 오프라인 ↔ VRX 대조

자산 — `UGV_Control_StateFlow_260529.slx` 의 4상태 미션 FSM (연구실 원본, 배포하지 않음)

10주차 · 킬링 추진기, 추력 배분과 동적위치유지 **신규**

제어 트랙의 마지막 주차. 9주차까지 **과소구동**이던 배를 **완전구동**으로 바꾼다.
원래 13주차에 있던 DP 를 여기로 앞당겼다 (13주차는 도킹 제어만 남는다).

이론 (50분)

- 과소구동 ↔ 완전구동 — 배분 행렬의 계급으로 판별
- 방위추진기(azimuth thruster) 와 $\pm 45^\circ$ 기계적 한계
- 확장 추력 벡터** — 비선형 배분을 선형으로 바꾸는 정식화
- 유사역행렬 해 $f = \text{pinv}(T_e) \cdot \tau$ 와 가중형
- 역추진 매핑 · 불감대 · 경계 클램프
- 도달가능 제어집합(ACS)** — 고정 추진기는 선분, 킬링은 면적
- 3자유도 **DP 제어법칙** $\tau = -J'(\psi)(K_p \cdot \eta_e + K_i \eta_e) - K_d \cdot \nu$
- 게인은 **대역폭**으로 정한다. $w_n = \sqrt{K_p/m}$
- 기준모델** — 계단 목표를 주면 포화한다
- 바람(상대풍·풍력계수) · 파랑(Bretschneider) 외란 모델
- 파랑 필터** — DP 대역폭 < 차단주파수 < 파랑 주파수

실습 (110분)

- `W10_alloc_demo.m` — ACS, $\pm 45^\circ$ 매핑, 기본 기동별 배분 그림 3장
- `W10_0_offline.slx` — 제자리 유지 → 전진 → **횡이동** → 제자리 선수 45° 회전 → 선수 0° 복귀 (5구간, 360 s)
- 파랑 필터 on/off 비교
- `W10_sway_demo.m` — VRX 개루프 횡이동 (되먹임 없이 옆으로 가는지 확인)
- `W10_1_vrx.slx` — VRX 연동 (`W10_vrx_run.m`). `thrusters/*/pos` 로 방위각 발행

i 실측으로 확인한 것 (2026-09-18, 바람 부호·클램프·Gain(-1)·서보 속도제한 수정 뒤)

- VRX 개루프 횡이동: 시뮬레이션 시각 25초에 **우현 23.99 m**, 전후 -1.45 m, 선수각 $+3.2^\circ$ (9주차까지는 0 m). 검사 $v_{ss} = 1.0$ m/s
- 오프라인 DP (바람 6 m/s + 파랑 Hs 0.3 m, 360 s): 구간 위치 RMS **0.069~0.107 m**, 선수각 RMS 0.92~2.11°, 최대 오차 1.977 m (η_d 기준)
- 파랑 필터: 추력 변화율 RMS **33.7 → 16.4 N/s (-51 %)**, 정착 후 위치 RMS 평균 0.079 → 0.083 m
- 기준모델: 전진 오버슈트 **2.84 → 1.65 m**, 추력 변화율 40.2 → 16.4 N/s (제어기·게인 동일). 대가는 도착 시각 (0.5 m 진입 8.6 → 20.2 s)
- VRX DP (360 s, 2026-09-19): 구간 위치 RMS **0.003~0.008 m**, 오프라인 대비 궤적 평균 이격 **0.16 m** (최대 0.86 m) — 네 주차 중 최소, 09-18 과 같음

✕ 부호 두 곳을 반드시 확인할 것

- NED 선체계에서 좌현 추진기는 $y_L = -1.027$ (ENU 의 $+1.027$ 이 아니다)
- `thrusters/*/pos` 는 ENU 관절각이므로 **NED 방위각과 부호가 반대**
둘 중 하나라도 틀리면 DP 가 선수를 못 잡고 발산한다. 실제로 겪었다.

과제 — 도달가능 제어집합 경계 손계산, 배분 검산표, DP 게인 3조건, 풍속 3조건 + 오프라인↔VRX 대조, 기준모델 on/off (검증 65%)

교재 — Fossen (2011) **12.3 Control Allocation** (특히 12.3.4 Azimuth Thrusters, p.408),
12.2.10 DP Case Study, 8.1 Wind Forces / Jin 외 (2025) Appl. Sci. 15:4213 식 (8)~(25)

자산 — `WAMV_USV_Control_Allocation.m`, `VRX_tilt4_controller_full.slx`

Phase III — 인지 트랙 (AI 에이전트 집중 활용 구간)

11주차 · LiDAR 신호처리: 포인트클라우드 전처리와 부표 탐지

이론 (50분)

- 3D LiDAR 측정 원리와 오차 요인
- `sensor_msgs/PointCloud2` 메시지 구조 (field, offset, point_step)
- 해수면 반사와 파랑에 의한 허위점 — 육상 로봇과 다른 점
- 복셀 다운샘플링
- RANSAC 평면 추정으로 수면 제거
- 유클리드 클러스터링 / DBSCAN, 클러스터 특징량 (크기, 점밀도, 반사강도)
- PointCloud2 → LaserScan 축약의 득실

실습 (110분)

- `sydney_regatta_ca.sdf` (부표 36개) 실행
- `/wamv/sensors/lidars/lidar_wamv_sensor/points` 로깅
- Claude 에이전트로 Open3D / PCL 전처리 파이프라인 작성 → 부표 클러스터 중심좌표 추정
- RViz2 마커 시각화
- `pointcloud_to_laserscan` 파라미터를 16빔 LiDAR에 맞게 재튜닝

과제 — 부표 탐지 성능 평가: 위치 RMSE, 오탐(FP)·미탐(FN), 거리 구간별(0~10 / 10~20 / 20~40 m) 검출률

자산 — `sydney_regatta_ca.sdf`, `pointcloud_to_laserscan/`

역할 로테이션 시점 — 이번 주부터 제어 담당과 인지 담당을 교체한다 (\$5.1 · [평가와-팀운영](#) 과 같은 11주차).

12주차 · LiDAR 기반 충돌회피

이론 (50분)

- VFH / VFH+ 극좌표 히스토그램 — 각도 구간별 장애물 밀도로 안전 방위 선정
- 동적창 접근법(DWA) — 속도 공간 탐색
- 속도장애물(VO) 개요
- 인공포텐셜장(APF)과 지역최소 문제
- 해상 적용의 한계: 관성이 크다, 횡류가 있다, 급정지가 안 된다, 최소선회반경이 존재한다
- COLREGs (국제해상충돌예방규칙) 개요

실습 (110분)

1. `collision_avoidance_v2.py` ($\pm 30^\circ$ 단순 회피)를 baseline으로 먼저 돌려 **실패 케이스를 눈으로 관찰**
2. `collision_avoidance_kaboat.py` 구조 분석 (슬라이딩윈도우 밀도, 비용 기반 방위 선정)
3. 에이전트와 함께 팀 자체 VFH 회피 노드 작성
4. 9주차 LOS 유도단과 결합 → 장애물 필드 통과 시험

과제 — baseline vs 자체 알고리즘 비교. 통과시간, 최소 이격거리, 경로길이, 충돌 횟수, 실패율. **각 조건 10회 반복** 후 평균·표준편차 제시

자산 — `vrx_control/collision_avoidance_*.py`, `vrx_kaboat.launch.py`

13주차 · 영상처리와 LiDAR-카메라 융합: 도킹 표식 인식

이론 (50분)

- 색공간 (RGB / HSV / Lab)과 해상 조명 변화·수면 반짝임 대응
- 형상 기술자: 윤곽 근사, Hu 모멘트

- YOLO 계열 검출기와 소량 데이터 학습 (개요)
- 카메라 - LiDAR 외부 파라미터 캘리브레이션과 점 → 이미지 투영
- 융합 수준 비교: early / late / hybrid fusion

실습 (110분)

- VRX 카메라 토픽 → HSV 임계 기반 부표 색상 분류 (적 / 녹 / 백)
- 도킹 표식 검출기 작성 — 삼각형 / 원형 / 사각형 × 색상
- LiDAR 클러스터에 카메라 색상 라벨 부여 → 목표 도크 선정 로직
- `scan_dock_deliver_full.sdf` 의 3개 베이에서 검증

과제 — 조명·파랑 3단계 조건에서 색상·형상 인식 정확도를 혼동행렬로 제시

자산 — `scan_dock_deliver_full.sdf`, WAM-V 카메라 3대 설정

Phase IV — 통합 Term Project

14주차 · 도킹 제어·미션 매니저와 통합 검증

이론 (50분)

- DP 는 10주차에서 이미 만들었다. 여기서는 그것을 **접안 절차에 끼워 넣는다**
- **접안 3단계**: 정렬(alignment) → 직진 접근(approach) → 정지(stop)
- 종료조건 판정 — 반경 기준 + 유지 시간 기준
- **Stateflow FSM vs Behavior Tree** — 언제 무엇을 쓰는가
- 실패 복구 동작과 E-stop

실습 (110분)

- `VRX_tilt4_controller_full.slx` 의 Stateflow 차트 분석 — **FSM 구조만 참고**하고 배분은 2추진기용을 씀

```
Unberthing      → Waypoint      [오차 ≤ threshold && |헤딩오차| ≤ 1 && duration(...) ≥ 5s]
Waypoint        → Dynamic_Positioning [waypoint_finished == 1]
Dynamic_Positioning → Docking_Approach [DP_error ≤ threshold && duration(...) ≥ 5s]
Docking_Approach → Docking_Final   [approach_finished == 1]
```

- `auto_berthing_seek.py` 의 **RANSAC 도크 평면** → 법선 → `/docking_info` 파이프라인 연결
- 팀 미션 매니저에 **충돌회피 상태를 삽입** — 어느 상태에서 회피가 활성화되어야 하는가

자산 — `VRX_tilt4_controller_full.slx`, `ros2_simulink/auto_berthing_seek.py`

이어서 · 통합 검증과 반복시험

DP 가 10주차로 옮겨 가면서 구 13·14주차를 한 주로 합쳤다.

추가 이론

- 실험 설계와 반복시험의 필요성 — 1회 성공은 성공이 아니다
- 성공률 · 소요시간 · 오차 지표 설계
- 로그 파이프라인: rosbag2 → CSV → 그래프
- 실패 모드 분류법
- Sim2Real 갭의 발생 원인: 모델 불일치, 센서 잡음 모델, 통신 지연, 추진기 응답

실습 (110분)

- **파랑·바람·초기조건을 랜덤화해 N회 자동 반복 실행하는 배치 스크립트 작성** (에이전트 활용에 적합)
- 로그 자동 수집·집계
- 실패 케이스 원인 분석
- 최종 데모 리허설

과제 — 팀별 성능 리포트: 임무별 성공률·소요시간·오차 지표를 표와 그래프로, **실패 원인 상위 3가지**와 대응책

15주차 · 최종 발표 및 데모

이론 (30분)

- 결과를 그림과 표로 말하게 하는 법
- 재현성과 코드 공개
- 향후 로드맵: KABOAT 출전, 실선 이식, Jazzy + Harmonic 이관

발표 (150분) — 팀별 25분 (발표 18분 + 질의 7분)

- 통합 데모 영상
- 알고리즘 설계 근거
- 정량 성능 지표
- 실패 분석
- AI 에이전트 활용 회고

최종 제출물

1. 최종 보고서
2. 통합 데모 영상
3. GitHub 레포 — README에 재현 절차 포함 (설치 → 빌드 → 실행 → 결과 확인)

5. 평가 방법

항목	비중	평가 기준
주차별 실습 과제 (12회)	25%	완성도보다 "측정하고 정량화했는가". 실패한 실험도 원인 분석이 있으면 만점
중간 발표 (8주차)	15%	전반부 데모 + Term Project 제안의 타당성·구체성
Term Project 데모·성능 (15주차)	35%	통합 성공률, 지표 달성도, 코드 품질(모듈화·재현성)
최종 보고서	15%	설계 근거, 결과 해석, 실패 분석의 논리
팀 기여도 (동료평가)	10%	팀원 상호평가 + 커밋·이슈 기록

5.1 팀 구성 및 운영

구성 — 4~5명 × 3~4팀. 팀장 선정.

역할	담당
인지 (Perception)	LiDAR·카메라 처리, 융합, 객체 라벨링
제어 (Control)	Simulink 제어기, 상태추정, LOS, DP
시뮬레이션 (Sim & Model)	월드·모델 수정, 파라미터, 배치 실험
통합 (Integration)	미션 매니저, 로그 파이프라인, 문서화

역할 로테이션을 11주차에 1회 강제한다. "인지만 아는 학생", "제어만 아는 학생"이 생기는 것이 캡스톤디자인에서 가장 위험한 실패 모드다.

미팅 — 팀 자체 주 1회 이상, 지도교수 미팅 2주 1회.
협업 도구 — GitHub Organization (팀별 레포 + 공용 패키지), Notion 또는 Google Drive.

6. AI 에이전트 활용 정책

원칙: 사용은 전면 허용한다. 평가하는 것은 "AI를 썼는가"가 아니라 "AI 출력을 검증했는가" 이다.

요구사항	내용
프롬프트 로그	주요 코드 생성에 사용한 프롬프트를 레포 docs/agent_log.md 에 기록
검증 기록	에이전트 초안 → 최종본 diff + 수정 사유. 5주차부터 매 과제 필수
설명 책임	발표 시 자기 코드의 임의 라인을 지목받아 설명할 수 있어야 한다. 설명 불가 시 해당 항목 감점
책임 소재	검증 없이 병합된 코드로 인한 데모 실패는 감점 사유로 인정한다 (변명 불가)

주차별 활용 지점

주차	활용 내용
5주	ROS 2 노드 스케폴딩, CLAUDE.md 작성, MATLAB MCP 연결
11주	포인트클라우드 전처리 파이프라인
12주	VFH 충돌회피 알고리즘 구현
13주	영상 기반 표식 검출기
14주	배치 실행 · 로그 집계 스크립트

7. Term Project 개요

- 상세 명세는 별도 문서 [Term-Project-명세](#) 참조.

미션 — 이안 → 장애물 필드 자율 통과 → 지정 부표 위치유지 → 목표 도크 식별 → 자율 도킹

구간	월드	요구 기술
① 이안·항주	sydney_regatta	Simulink 헤딩/속도 제어 + LOS
② 장애물 회피	sydney_regatta_ca (부표 36개)	LiDAR 클러스터링 + VFH / DWA
③ 위치유지	지정 부표	DP, 반경 5 m 내 5초 유지
④ 도킹	scan_dock_deliver_full (3 베이)	표식 인식 + RANSAC 도크 평면 + 접안 제어

평가 지표 (각 10회 반복) — 완주 성공률, 총 소요시간, 장애물 최소 이격거리, 충돌 횟수, DP 위치오차(평균·표준편차·최대), 도킹 위치오차·헤딩오차·성공률

가산점 옵션 — 파랑·바람 3단계 외란 조건, 도크 베이 무작위 지정, 장애물 배치 무작위화

8. 개강 전 준비 체크리스트 (담당 교수 / 조교용)

#	항목	우선도	상태
1	MATLAB ROS Toolbox ↔ ROS 2 Humble 실통신 검증 — 개강 2주 전 pub/sub 왕복 확인. 실패 시 Simulink 코드 생성 → ROS 2 노드 빌드로 우회 (<code>vrx_control_thruster_waypoints/</code> 에 선례 있음)	High	☐
2	WSL 이미지 사전 배포 — Ubuntu 22.04 + Humble + Garden + VRX 프리빌드 상태를 <code>wsl --export</code> 로 tar 배포. 개별 설치 시 VRX <code>colcon build</code> 자체는 39.7초(데스크톱)~1분 54초(노트북) 실측이나 apt-rosdep 내려받기가 네트워크에 따라 길어짐	High	☐
3	학생 노트북 사양 사전조사 — RAM 16 GB / 4코어 미달자용 연구실 워크스테이션 원격 접속(SSH / NoMachine) 준비	Med	☐
4	MATLAB 라이선스 동시접속 수 확인 — 팀당 1석 운영 전제	Med	☐
5	설치 자료 오류 수정판 배포 — WSL-VRX-환경구축 에 반영 완료	Med	☒
6	하드코딩 경로 정리 — <code>VRX_berthing_disturbance_script.m</code> 의 <code>addpath("C:\Users\ANSL\Documents\MATLAB")</code> , <code>VRX_SHIFT_MINI_Full.m</code> 의 <code>circle_draw</code> 로컬 함수 새도임	Low	☐
7	GitHub Organization 개설 및 팀 레포 템플릿 준비	Med	☐
8	커스텀 월드 배포 패키지 제작 — 월드 4개 + <code>competition4docking.launch.py</code> + <code>custom_docking_station</code> 은 upstream VRX 에 없음. 11주차 전까지 tar 로 묶어 배포 → VRX-월드와-패키지	High	☐
9	Claude 계정 확보 — Claude Code 는 무료 플랜에서 동작하지 않음(Pro-Max-Team-Enterprise 또는 Console 필요). 5주차 전까지 학생 계정 방식 결정: 개인 구독 / 연구실 공용 / API 키 배정	High	☐
10	MATLAB Agentic Toolkit 사전 검증 — <code>agenticToolkitInstaller.mltbx</code> 설치 후 <code>setupAgenticToolkit("install")</code> → <code>shareMATLABSession()</code> 로 Claude 연결 1회 확인. 담당 PC 에서는 확인 완료(2026-09-03)	Med	☐
11	<code>custom_docking_station</code> 모델 위치 확인 — <code>vrx_urdf/vrx_gazebo/models/</code> 에 존재 확인 완료	—	☒

9. 명령어 검증 이력

- 2026-09-03 · WSL Ubuntu 22.04 + ROS 2 Humble + VRX 2.4.1 실행환경에서 검증

대상	방법	결과
apt 패키지명 23종	<code>apt-cache policy</code>	전부 실재
<code>ros2</code> CLI 옵션 (<code>--rate</code> <code>--verbose</code> <code>--license</code> <code>--build-type</code> <code>hz</code> <code>bw</code> <code>bag</code> <code>param</code>)	<code>--help</code> 대조	전부 유효
실행파일 (<code>view_frames</code> <code>tf2_echo</code> <code>static_transform_publisher</code> <code>talker</code> <code>listener</code> <code>rqt_graph</code> <code>rviz2</code> <code>gz</code>)	<code>ros2 pkg executables</code>	전부 존재
<code>colcon build</code> 옵션 4종	<code>--help</code> 대조	전부 유효
rcldpy QoS · 메시지 타입 임포트	실제 <code>import</code>	전부 성공
<code>git checkout humble</code>	<code>git ls-remote --heads origin humble</code>	브랜치 실재 (<code>dc30ed8d</code>)
<code>competition.launch.py</code> 인자 (<code>world</code> <code>headless</code>)	<code>--show-args</code>	유효
토픽 8종 (GPS · IMU · LiDAR points/scan · 추진기 thrust/pos ×2)	VRX 헤드리스 기동 후 <code>ros2 topic list</code>	전부 실재
커스텀 월드 4종 · <code>competition4docking.launch.py</code>	<code>git ls-tree HEAD</code> 대조	upstream 에 없음 → 배포 필요
MATLAB(Windows) ↔ WSL ROS 2 통신	실제 노드 띄우고 MATLAB 에서 수신	NAT 모드에서 그대로 동작
Simulink → Gazebo 구동	<code>W06_1_straight.slx</code> 실행 후 odometry 확인	성공 — u 가 0.0008 → 1.33 m/s
ground truth odometry	urdf 의 <code>xacro</code> 인자 <code>ground_truth_enabled</code> 를 true 로 고쳐 <code>urdf:=</code> 로 기동 (런치 인자로 주면 무시됨 — 2026-09-15 확인)	<code>/wamv/sensors/position/ground_truth_odometry</code>
Simulink 모델 4종 (6주차)	컴파일 + 미연결 포트 검사	전부 통과

7주차 준비 과정에서 추가로 확인한 것

대상	방법	결과
VRX 해류 플러그인	레포 전체 <code>grep -rniE 'ocean_current water_current tidal drift'</code>	0건 — 해류가 없다. <code>SimpleHydrodynamics</code> 가 항력을 대지속도로 계산
VRX 바람	<code>sydney_regatta.sdf</code> 확인 + 무추력 표류 실측	기본 월드는 풍속 0. <code>2023_practice/practice_2023_wayfinding{0,1,2}_task</code> 가 0 / 4 / 8 m/s. 25초에 2.17 m 표류 확인
바람 런타임 변경	<code>--show-args</code> , <code>USVWind.cc</code> 의 <code>Subscribe</code> 수	불가능 — 인자도 토픽도 없음. world 를 바꾸는 수밖에
추진기 입력 단위	플러그인 심볼 + 정상상태 대조	뉴턴(N). PWM 모델 없음. <code>use_angvel_cmd</code> 미설정
추진기 최대 추력	<code>wamv_gazebo_thruster_config.xacro</code> 계산	$((51.3 + 72.4 \times 7.71667) \times 7.71667) / 2 = **2353.6 \text{ N**}$
프로펠러 상수	$p \cdot Ct \cdot D^4 = 1000 \times 0.004422 \times 0.2^4$	$0.0070752 \text{ N/(rad/s)}^2$
유체력 계수	<code>wamv_gazebo_dynamics_plugin.xacro</code>	<code>Xu=100, Xuu=150, Yv=100, Yvv=100, Nr=800, Nrr=800</code> , 부가 질량 전부 0
3자유도 축소모델 타당성	정상상태 $400 = (100+150u)u$ vs Gazebo 실측	$u = 1.333$ 계산 / 1.331 실측
요각속도 부호	<code>FL=+250, FR=-250</code> 인가 후 <code>twist.angular.z</code>	$-0.443 \rightarrow r_NED = -\omega_z^{ENU}$ 확인
실시간 계수 (RTF)	odometry 타임스탬프 vs 벽시계 20초	0.476 — 페이싱을 이 값에 맞춰야 함
스폰 위치	<code>competition.launch.py</code> + 실측	설치마다 다름 — 같은 VRX 2.4.0-2 인데 ENU $(-532, 162)$ (노트북, 2026-09-15) / $(-532, 200)$ (데스크톱, 2026-09-18), yaw 1.0 rad $\rightarrow \psi \approx 32.9^\circ$. 고정값을 틀리게 쓰면 경로가 38 m 밀려 안전 수역을 벗어남 $\rightarrow W07 \sim W10$ <code>vrx_run</code> 이 ground truth 첫 샘플로 원점을 실측해 덮어씀
안전 수역	<code>wayfinding_task.sdf</code> 웨이포인트 3점 환산	스폰 기준 $x(\text{북}) \in [-28, 60]$, $y(\text{동}) \in [-18, 100]$
오프라인 모델 ↔ Gazebo 일치도	동일 게인·초기조건 150초 주행	정상상태 u 1.500 vs 1.500, 평균
Simulink 모델 2종 (7주차)	컴파일 + 미연결 포트 검사 + 실주행	전부 통과

8·9주차 준비 과정에서 확인한 것

대상	방법	결과
로이터 벡터필드 회전 방향	논문 III절 원문 + 정복 지점 속도 부호	$p_c > 0$ 이 시계방향. 참고 코드의 한글 주석이 반대로 적혀 있음
p_c 설계식	논문 식 (27)·(28) 역산 + 실측 대조	$p_c = 4\zeta_c^2 v r_d / r_d$. 논문의 0.33 은 $v=25, r_d=150, \tau_r=0.5$ 기준
반경 정상상태 오차	p_c 4점 스윙	$r_{ss} - r_d = p_c \cdot v \cdot \tau_{eff}$, 오차 ÷ p_c = 6.82 로 일정
오차의 분해	이론 예측 vs 실측	헤딩 램프 2.50 + 모터 0.30 + 크랩각 1.93 = 4.73 s (실측 4.55)
크랩각 보상 효과	$crab_comp$ 0 ↔ 1	2.137 → 1.244 m (이론 예측 1.31 m)
방향·속도·반경 변경	시나리오 4단계	전부 정상. 속도 2.4 목표 시 추력 포화로 2.18
WAM-V 최소 선회반경	$N_{max} = 2F_{max} \cdot b = 1027 N \cdot m$	2.03 m ($u=1.5$). 반경 25 m 는 여유 있음
미션 FSM (9주차)	오프라인 전 구간 주행	진입 74.7 s → 종료 248.0 s (2.00 바퀴) → 임무 종료 314.4 s (2026-09-18 재측정)
대수 루프	컴파일 오류 재현 후 해결	$mode$ 되먹임에 Unit Delay 1개 필요
Simulink 모델 4종 (8·9주차)	컴파일 + 미연결 포트 검사 + 실행	전부 통과, 미연결 0

VRX 실연동 검증 (2026-09-04 최초, 2026-09-18 · 2026-09-19 재측정)

7~10주차 네 모델을 실제로 Gazebo 와 연동해 돌리고 오프라인과 대조했다.

재측정 환경: 데스크톱, WSL Ubuntu-22.04, VRX 2.4.0-2, sydney_regatta, ground_truth_enabled=true, 스폰 ENU (-532, 200), 실험마다 VRX 새로 띄움, gz RTF 0.96~0.99 (09-18), 페이싱 실측 RTF 0.67~0.70.

오프라인은 VRX 와 같은 초기 선수각으로 다시 돌린 값과 대조한다. 본문 수치와 그림은 2026-09-19 실행.

주차	모델	시간	궤적 평균 이격 09-18	09-19 (최대)	판정 지표 09-19 (VRX / 오프라인)
7	W07_1_vrx	230 s	0.28 m	0.95 m (1.85 m), 같은 날 다른 실행 2.59 m	정착 후 $ y_e $ 0.784 / 0.702 m, u 1.501 / 1.500 m/s
8	W08_1_vrx	300 s	4.96 m	0.77 m (1.96 m)	반경오차 2.265 / 2.138 m
9	W09_1_vrx	420 s	0.83 m	4.78 m (14.44 m)	반경오차 1.213 / 1.282 m, 상태 순서 같음
10	W10_1_vrx	360 s	0.16 m	0.16 m (0.86 m)	평균 위치 오차 0.313 / 0.433 m

- 2026-09-04 의 값(1.11 / 0.31 / 5.07 / 0.25 m)은 스폰 원점 실측 등 코드 수정 이전 값이라 폐기
- 7~9주차 궤적 평균 이격은 실행마다 크게 달라짐 — 페이싱 비율(측정 RTF)과 실제 진행이 어긋난 만큼 같은 길을 다른 시각에 지나는 위상 오차. 7주차 같은 설정 세 번: 0.28 · 2.59 · 0.95 m, 정착 후 $|y_e|$ 는 0.785 · 0.779 · 0.784 m, u 는 셋 다 1.501 m/s
- 판정은 이격이 아니라 정착 후 $|y_e|$ · 반경오차 · 정상상태 속도로 함 → 궤적이-겹치면-이격은-오차가-아니라-지연이다
- 10주차가 가장 잘 겹침 — DP 는 제자리에 머물러 위상 오차가 드러나지 않고, 속도가 느리고 과도응답이 적음
- 9주차 09-19 실행은 VRX 가 9.5 s 빨리 끝남 (임무 종료 301.1 vs 310.6 s). 궤적 그림에서 두 선은 겹침
- W0X_vrx_run 의 초기 선수각 읽기 수정 (2026-09-19): 0.5 s 시점 값을 쓰다 odom 이 늦게 오면 0° 를 읽었음 → 처음으로 0 이 아닌 선수각을 씀
- 개인은 네 주차 모두 오프라인에서 맞춘 값 그대로. 하나도 바꾸지 않았다

새로 만든 것

파일	역할
W07_vrx_run.m ~ W10_vrx_run.m	토픽 확인 → 스폰 원점 실측 → RTF 자동 측정 → 페이싱 → 실행 → 결과·대조 그림
VRX 모델의 Animate 블록	주행 중 실시간 항적을 그린다 (7주차 모델에 새로 추가)

고친 결함

증상	원인	조치
로그 첫 점이 568 m 됨	Subscribe 가 첫 메시지 전에는 0 버스를 냄 → <code>x_n = 0 - 200</code> , <code>y_n = 0 + 532</code>	Nav 첫머리에 <code>if ex==0 && ey==0</code> 가드
대조 그래프가 폭주	두 모델의 종료 시각이 달라 <code>interp1</code> 외삽	겹치는 시간 구간만 비교
초기 선수각이 90° 로 잡힘	위와 같은 원인의 첫 샘플	<code>t ≥ 0.5 s</code> 샘플에서 읽음

⚠ pkill 패턴을 스크립트 안에서 쓸 때

`pkill -f "vrx_gz|..."` 를 `bash -lc "..."` 안에 넣으면 자기 자신의 명령줄이 패턴에 걸려 셸이 먼저 죽는다. 스크립트에서는 `[v]rx_gz` 처럼 써야 한다.
터미널에 직접 칠 때는 문제없다.

10주차 검증 (2026-09-04, 2026-09-18 재측정)

- 2026-09-18 에 바람 부호·클램프·`Gain(-1)` ·서보 속도제한을 고친 뒤 수치를 다시 잼. 아래 표는 재측정값

대상	방법	결과
VRX 방위추진기 동작	<code>thrusters/*/pos</code> 로 각도 지령 후 개루프 25 초 (시뮬레이션 시각)	우현 23.99 m, 전후 -1.45 m, 선수각 +3.2°. 검산 $v_{ss} = 1.0$ m/s. 예전 10.86 m 는 벽시계 기준이라 폐기
pos 부호	두 차례 개루프 실험으로 확인	NED 방위각과 반대. Gain(-1) 필요
T_e 요 행 부호	방위각 0 에서 7주차 식과 대조 (<code>assert</code>)	좌현은 NED 로 <code>y_L = -1.027</code>
배분 계산	<code>T_e · f</code> 와 요구 τ 대조	전진·횡이동·선회 모두 정확히 일치
도달가능 제어 집합	몬테카를로 + convex hull	고정: X ± 1000 , Y 0 / 틸팅: X ± 1000 , Y ± 707
기준모델 효과	on/off 비교	전진 오버슈트 2.84 → 1.65 m, 횡이동 오버슈트 2.49 → 1.62 m, 추력 변화율 40.2 → 16.4 N/s
파랑 필터 효과	on/off 비교	추력 변화율 33.7 → 16.4 N/s (-51 %), 방위각 변화율 9.21 → 7.72 °/s, 위치 0.079 → 0.083 m
오프라인 DP	5구간 시나리오 360 s, 바람 6 m/s + 파랑 Hs 0.3 m	위치 RMS 0.069~0.107 m, 선수각 RMS 0.92~2.11°, 최대 오차 1.977 m (η_d 기준)
VRX DP	360 s, 페이싱 실측 RTF 0.679 (2026-09-19)	위치 RMS 0.003~0.008 m, 궤적 평균 이격 0.16 m (최대 0.86 m)
파랑 필터 안정성	차단주파수를 DP 대역폭 아래로	한계진동 발생 (진폭 40°, 주기 40 s) — 대역폭 관계를 문서에 명시

6주차 오프라인 모델 검증 (2026-09-04)

항목	오프라인	VRX	차이
직진 정상상태 u [m/s]	1.333	1.332	-0.001
우선회 r [deg/s]	+14.65	+15.72	+1.07 (7.3 %)
좌선회 r [deg/s]	-14.65	-15.74	-1.09
우선회 변경 [m]	4.61	4.09	-0.52

- 직진은 소수점 셋째 자리까지 일치. 선회는 7 % 차이 — 요 방향의 미포함 향이 남아 있음
- `w06_vrx_record.m` 이 VRX 를 시뮬레이션 시각 기준으로 구동해 기록한다 (RTF 무관)

모델 배치 정리 (`_tools/tidy_layout.m`)

- 목적 — 학생이 처음 열었을 때 선을 눈으로 따라갈 수 있게 하는 것
- 대상 — 6~9주차 Simulink 모델 22개 전부

항목	결과
블록 겹침	659개 블록 중 0쌍
선	667개 중 꺾인 선 97개 (14.5%) — 나머지는 완전 직선
대각선	0개
색	유도 파랑 · 미션 보라 · 제어 주황 · 추진기 노랑 · 운동모델 초록 · ROS 연보라 · 로깅 회색

- 꺾인 선은 **되먹임**(Unit Delay 되돌림)과 포트 개수가 다른 블록 사이에만 남음
- 서브시스템 내부는 `Simulink.BlockDiagram.arrangeSystem` 으로 정리 — 직각 배선, 겹침 0
- `build_wXX_models.m` 마지막에서 자동 호출되므로 **재생성해도 배치가 유지됨**
- 학생 배포 폴더 5곳에 `tidy_layout.m` 사본을 넣어 두어 `tidy_layout('W07_0_offline')` 한 줄로 복구 가능

✕ 정리 명령에 `vrx_ros` 를 넣을 것

```
kill -f "vrx_gz|vrx_ros|ros_gz_bridge|gz sim|ruby|parameter_bridge"
```

기존 패턴으로는 `optical_frame_publisher` · `pose_tf_broadcaster` 가 안 잡혀 고아 프로세스로 남는다.

본 계획서는 「WSL를 이용한 VRX 환경 구축.pptx」, 「2025 KABOAT 임무설명_r1.docx」, Jin et al. (2025) Appl. Sci. 15(8):4213, 그리고 [2025] ROS2_VRX_Gazebo_Simulink 워크스페이스의 실제 자산 조사를 근거로 작성되었다.